

Marine Race Arena: A Configurable HoloOcean Benchmark for Underwater Gate Racing and Team-Level Fleet Evaluation

Andrea Bedei, Lorenzo Bacchiani, Giovanni Pau and Roberto Girau
Department of Computer Science and Engineering, University of Bologna, Italy

Abstract—We present Marine Race Arena, a configurable and reproducible framework for developing and evaluating autonomous systems in underwater gate-racing tasks. Complete race scenarios can be defined through declarative configurations, including tracks, vehicles, sensors, currents, obstacles, rules, penalties and starting conditions. The benchmark strictly separates autonomy from evaluation: controllers operate exclusively from onboard sensors, simulated acoustic beacons and optional inter-vehicle communication, while an independent referee uses privileged simulator state only to validate gate crossings, enforce the rules and compute the final results. The framework supports both single-vehicle trials and cooperative multi-vehicle scenarios, while providing a simple interface for integrating custom controllers. The release includes three ready-to-use tracks, two camera- and beacon-assisted reference controllers, leader-follower coordination tools, structured logging and automated validation procedures. Experiments conducted in HoloOcean demonstrate the ability of the benchmark to consistently evaluate course completion, timing, collisions, robustness to currents and fleet coordination. Marine Race Arena therefore provides a common platform for comparing underwater perception, navigation, control and cooperation strategies under repeatable and progressively more challenging conditions.

Index Terms—underwater robotics, autonomous racing, HoloOcean, BlueROV2, simulation benchmark, referee, multi-robot systems, team-level evaluation

I. INTRODUCTION

Autonomous racing has emerged as an effective research format for evaluating perception, planning, control and real-time decision-making within a single quantitative task under repeatable conditions [1]. In ground robotics, the F1TENTH platform provides standardized vehicles, circuits, metrics and baselines for continuous control and reinforcement learning [2]. At full scale, competitions such as the Abu Dhabi Autonomous Racing League (A2RL) push planning and control toward the limits of vehicle handling in head-to-head racing scenarios [3]. In aerial robotics, gate-based benchmarks and competitions have driven progress from modular perception and control pipelines [4] to learning-based policies capable of champion-level drone racing [5]. Across these domains, the availability of a well-defined racing task, fixed course geometry, explicit scoring rules and machine-readable outcomes has made autonomy methods easier to compare, reproduce and improve.

Underwater robotics has not yet benefited from an equivalent benchmark for speed-oriented autonomous racing along

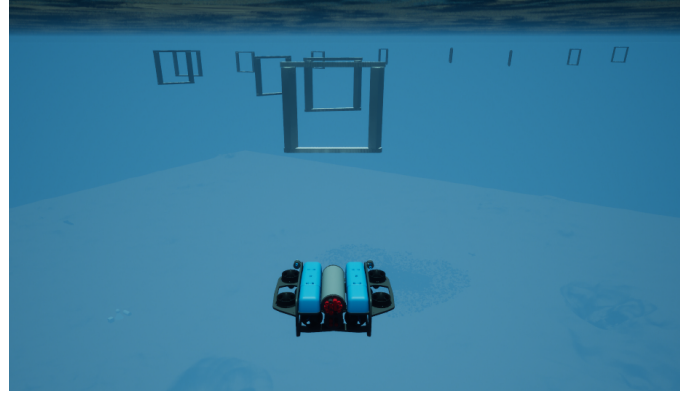


Fig. 1. Underwater gate racing in HoloOcean. A vehicle follows the ordered, beacon-marked gates of the Horseshoe Bay circuit using onboard sensing.

predefined gate courses. Existing marine simulators provide increasingly mature support for underwater vehicles, sensors and environments, but they mainly target intervention, inspection, navigation or sensor realism rather than racing as an evaluation protocol. In particular, they do not provide a compact underwater counterpart to ground and aerial racing benchmarks: a task defined by ordered gates, timing, penalties, currents, obstacles, independent scoring and reproducible outcomes. This gap matters because underwater racing is not simply aerial or ground racing transferred to a different simulator. Underwater vehicles are slow, strongly damped and affected by persistent currents; global localization is typically unavailable; inertial and Doppler sensing may drift; optical perception is limited in range and degraded by turbidity; acoustic cues are noisy and low bandwidth; and physical contact with gates, obstacles or other vehicles can be difficult to interpret. These properties change both the information available to a controller and the way in which performance should be evaluated.

A central requirement for such a benchmark is therefore a strict separation between autonomy and privileged simulation services. A controller should be evaluated using only information that would plausibly be available onboard, such as local sensing, physically received acoustic packets and controller-local state. Privileged simulator state, including global pose, exact gate geometry, official course progress and the true current field, must remain outside the controller boundary. The benchmark may use it internally to construct the phys-

ical environment and referee inputs, but official progress is evaluator-only and none of this state flows back to autonomy. Without this boundary, performance can depend on information unavailable to a real vehicle, making results difficult to interpret and comparisons unfair.

This paper presents Marine Race Arena, a configurable benchmark layer for autonomous underwater gate racing (Fig. 1). Its benchmark model, controller contract and referee are vehicle-independent; replaceable adapters isolate simulator-specific sensing and actuation. The reference implementation and all reported experiments use HoloOcean [6] with a BlueROV2-class vehicle [7]. Rather than replacing a simulator or proposing an autonomy algorithm, Marine Race Arena defines an evaluation contract on top of a simulation backend. A JSON configuration specifies the world, ordered gates, sensors, currents, obstacles and scoring rules. Controllers receive only an official observation, while an independent referee uses privileged state to validate crossings and violations, assign penalties, manage timeouts and compute official scores.

The core task evaluates a single autonomous vehicle whose controller is integrated through a small interface (Section V). Fleet mode extends the same mechanism to a cooperative team in which each vehicle runs its own controller and retains independent progress. This separation from evaluation allows the benchmark to compare arbitrary controllers, including future learning-based ones, under the same official observation.

The contributions of this paper are as follows.

- 1) We present Marine Race Arena, an end-to-end configurable benchmark for autonomous underwater gate racing. Users can define complete race scenarios, including the environment, vehicle configuration, sensor profile, ordered gates, currents, obstacles, timing and scoring through a single configuration file. The release includes three ready-to-use official tracks and supports the creation of custom courses and experimental scenarios.
- 2) We provide an extensible autonomy interface for controller development and comparison. The framework includes two camera-assisted rule-based reference controllers and allows users to integrate their own controller through a minimal three-method interface without modifying the race runner, simulator adapter or evaluation logic. A documented official observation contract ensures that every controller is evaluated using the same non-privileged information.
- 3) We introduce an independent automated referee that uses privileged simulator state only for evaluation. It validates ordered gate crossings, detects rule violations, applies penalties and timeouts, ranks participants and produces structured event logs and machine-readable race summaries.
- 4) We support both single-vehicle racing and cooperative multi-vehicle evaluation within the same framework. Each vehicle maintains independent control and a locally estimated course state, while the referee maintains separate official scoring state for the team result. The fleet

layer provides staggered or simultaneous starts, inter-vehicle proximity diagnostics, optional acoustic communication and distributed leader-follower coordination.

- 5) We provide reproducible experimental validation in HoloOcean covering the three official tracks, a comparison between the two reference controllers, disturbance-current scenarios and multi-vehicle coordination. The accompanying configurations, scripts and structured outputs allow the reported experiments to be reproduced and extended with new tracks, vehicles and control strategies.

The remainder of the paper is organized as follows. Section II reviews underwater simulators, autonomous-racing benchmarks and multi-AUV underwater cooperation. Section III formalizes the benchmark model and runtime architecture. Section IV describes the vehicle, the HoloOcean adapter, the sensor suite, the current model and the official observation contract. Section V specifies the controller interface and the rule-based baseline. Section VI formalizes gate validation, the referee state machine and single-vehicle scoring. Section VII describes staggered fleet execution and team scoring. Section VIII reports the experimental evaluation. Section IX discusses limitations and future work. Section X concludes the paper.

II. RELATED WORK

We review related work along three axes: underwater simulators that provide the physical and sensing substrate, autonomous-racing benchmarks that define the evaluation format we adapt and literature on multi-AUV cooperation and acoustic communication that motivates and bounds the fleet mode. Table I summarizes the positioning of Marine Race Arena against representative systems.

A. Underwater Robotics Simulators

Several simulators provide marine vehicles and sensors. The UUV Simulator offers a Gazebo-based package for underwater intervention and multi-robot scenarios [8]. Stonefish provides custom marine-robotics physics and a wide range of underwater sensors behind a ROS interface [9]. DAVE builds on Gazebo to support acoustic and optical sensing for autonomous underwater vehicles [10]. HoloOcean uses Unreal Engine to render underwater scenes and exposes common marine sensors, including sonar, an RGB camera, an IMU, a DVL and a depth sensor, for one or more agents [6]. It is the simulation backend used in this work. MarineGym extends Isaac Sim with GPU-accelerated hydrodynamics, multiple UUV models, domain randomization and Gym-style benchmarks for station keeping, trajectory tracking and docking [11]. MarineGym targets scalable reinforcement-learning training and standardized underwater-control tasks, whereas Marine Race Arena defines a gate-racing evaluation protocol. The other simulators primarily supply vehicles, physics and sensors. Marine Race Arena complements both categories by adding the race task, official observation contract, referee, scoring and output protocol. None provides an underwater gate-racing benchmark

TABLE I
COMPARISON WITH REPRESENTATIVE UNDERWATER SIMULATORS AND RACING BENCHMARKS.

System	Domain	Physics and sensing	Racing task	Multi-robot	Indep. referee	Official obs. contract
UUV Simulator [8]	Underwater	Gazebo, hydrodynamics	no	✓	no	no
Stonefish [9]	Underwater	Custom physics, marine sensors, ROS	no	no	no	no
DAVE [10]	Underwater	Gazebo, acoustic and optical sensing	no	✓	no	no
MarineGym [11]	Underwater	Isaac Sim, GPU hydrodynamics and RL tasks	no	partial	no	✓
F1TENTH [2]	Ground	Scaled car, LiDAR, RL and control	✓	partial	partial	✓
AlphaPilot and Swift [4], [5]	Aerial	Quadrotor, vision-based gate racing	✓	no	partial	✓
CARLA [12]	Ground	Unreal Engine, urban sensing, tasks	partial	no	partial	✓
Marine Race Arena (this work)	Underwater	HoloOcean, BlueROV2, acoustic and camera sensing	✓	✓	✓	✓

“Partial” marks a supported feature that is not a primary design contract.

with an impartial referee or enforces an information boundary between the controller and ground-truth state.

B. Autonomous Racing Benchmarks and Competitions

Racing benchmarks are valuable because they expose end-to-end behavior under repeatable, adversarial constraints, as catalogued by the survey of Betz et al. [1]. F1TENTH provides a scaled autonomous car, standardized circuits and an evaluation environment for continuous control and reinforcement learning [2]. It is the closest in spirit to Marine Race Arena, but targets ground vehicles with planar LiDAR and treats referee logic as a partial, in-framework concern rather than a separated, first-class contract. At full scale, the A2RL challenge frames racing as multi-agent interaction at the limits of vehicle handling [3], while the A2RL league also runs a drone-racing format [13]. In aerial robotics, AlphaPilot established gate sequences as a demanding benchmark for fast perception and control [4] and Swift later demonstrated champion-level drone racing using deep reinforcement learning trained against a simulated opponent [5]. CARLA is not a racing benchmark, but illustrates how an Unreal-based ecosystem can standardize maps, sensors and tasks for driving research [12]. More broadly, standardized environment interfaces such as OpenAI Gym [14] and GPU-parallel robot-learning simulators such as Isaac Gym [15] show how a documented observation-action contract enables systematic comparison of learning-based controllers. Marine Race Arena adopts the gate-racing task and documented observation contract from these efforts, adapting them to underwater sensing, slow and damped vehicle response, acoustic guidance, current disturbances and a referee whose scoring remains entirely outside the control loop.

C. Multi-AUV Cooperation and Underwater Communication

Fleet operation underwater is constrained by the physics of the acoustic channel and by uncertain localization. Underwater acoustic links suffer from low bandwidth, long propagation delay, multipath and time-varying attenuation [16], so the assumption of continuous high-bandwidth, low-latency inter-vehicle communication that underpins many terrestrial multi-robot methods does not transfer. Surveys of AUV formation

control document how these communication and navigation constraints shape achievable coordination: Yang et al. analyze formation performance, control and communication capability jointly [17] and Yan et al. review formation-control methods and their dependence on relative-state estimation [18]. Consistent with this literature, Marine Race Arena does not assume shared ground-truth state or high-rate communication between vehicles. Its fleet mode runs multiple HoloOcean vehicles as one cooperative team with staggered or simultaneous releases, independent per-vehicle onboard autonomy and referee state, inter-vehicle proximity monitoring, team-level scoring, an acoustic-inspired inter-vehicle channel and a distributed leader-follower coordination controller. The channel supports coordination but is not characterized as a standalone communication study.

D. Positioning

In contrast to the systems above, Marine Race Arena is the only one that simultaneously provides an underwater simulation backend, a gate-racing protocol with timing, an independent referee that uses privileged state, multi-vehicle and team-level scoring and an enforced official-observation contract that excludes ground-truth pose from control (Table I). It does not compete with the underwater simulators on physical fidelity or with the racing platforms on autonomy performance; it complements both by supplying the missing evaluation contract for underwater gate racing.

III. BENCHMARK MODEL AND SYSTEM ARCHITECTURE

We specify Marine Race Arena as an *evaluation problem* rather than a controller-design problem: the benchmark fixes the world, task and scoring, while a participant supplies only a policy. This section defines the benchmark instance, participant interface and runtime architecture that enforce the separation between the two.

A. Benchmark Instance

Definition 1 (Benchmark instance). A Marine Race Arena instance is the tuple

$$\mathcal{E} = (\mathcal{W}, \mathcal{G}, \mathcal{S}, \mathcal{F}, \mathcal{C}, \mathcal{O}, \mathcal{P}, \mathcal{R}), \quad (1)$$

where:

- $\mathcal{W}$ is the *world*: an axis-aligned bounds box $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}] \subset \mathbb{R}^3$ (with z positive-up, so the navigable volume satisfies $z < 0$) and the simulation environment selected through the adapter;
- $\mathcal{G} = (g_1, \dots, g_M)$ is the *ordered* gate sequence; each gate is the tuple $g_k = (c_k, n_k, w_k, h_k)$ of center $c_k \in \mathbb{R}^3$, passage-direction vector $n_k \in \mathbb{R}^3$ (nonzero, not necessarily unit length) and inner aperture width w_k and height h_k ;
- $\mathcal{S}$ is the shared *start default*: a base spawn pose and an optional release policy that a participant inherits only when it does not specify its own;
- $\mathcal{F}$ is the finish condition: completion of the last gate of the last lap (Section VI);
- $\mathcal{C}$ is the current field $\mathbf{v}_c : \mathbb{R}^3 \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^3$ (Section IV);
- $\mathcal{O}$ is the obstacle set;
- $\mathcal{P} = \{1, \dots, N\}$ is the participant set; participant i carries a vehicle, sensor profile and controller, together with an optional spawn pose and release delay $\tau_i \geq 0$ that override the start default of $\mathcal{S}$;
- $\mathcal{R}$ is the referee configuration: gate-validation parameters, penalty values and scoring rules.

Each gate’s orientation derives solely from its passage direction n_g , giving a right-handed frame

$$\hat{n} = \frac{n_g}{\|n_g\|}, \quad \hat{r}_g = \frac{\hat{z} \times \hat{n}}{\|\hat{z} \times \hat{n}\|}, \quad \hat{s}_g = \hat{n} \times \hat{r}_g, \quad (2)$$

with world-up $\hat{z} = (0, 0, 1)$; if the passage direction is vertical ($\|\hat{z} \times \hat{n}\| \approx 0$), $\hat{r}_g$ falls back to the world $\hat{x}$ axis so the frame stays well defined. The gate basis is therefore unambiguous and independent of any visual rotation and is reused by the beacon placement (Section IV) and the crossing test (Section VI).

B. Participant Interface

Definition 2 (Policy). A participant i implements a policy

$$\pi_i : o_{i,t} \mapsto u_{i,t}, \quad (3)$$

mapping the official observation $o_{i,t}$ (Section IV) to a normalized, high-level, body-frame command

$$u_{i,t} = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^\top \in [-1, 1]^4. \quad (4)$$

The action of (4) is the portable, constrained interface that an adapter maps to the selected vehicle. In fleet mode the policy may return $(u_{i,t}, m_{i,t})$, where $m_{i,t}$ is an optional controller-authored communication payload and only $u_{i,t}$ is a physical action. The reference HoloOcean adapter also accepts a direct eight-thruster `thrusters` action for its BlueROV2-class agent. Time is discretized at a fixed control timestep Δt , with $t_k = k \Delta t$. The adapter clamps $u_{i,t}$ to actuator limits and applies depth-bound safety before mapping it to the vehicle; in official mode the policy reads only the official observation $o_{i,t}$, whose contract excludes privileged state (Section IV).

TABLE II
DESIGN REQUIREMENTS FOR THE BENCHMARK LAYER.

ID	Requirement
R1	Race configuration is declarative and reproducible from a single set of track JSON files.
R2	Official controller observations exclude ground-truth pose, simulator-only state and referee-derived course progress.
R3	Referee logic is independent of the participant controller and cannot be influenced by it.
R4	Gate dimensions, referee margins and track geometry are not changed to hide failures.
R5	Every run emits structured event logs and machine-readable summaries.
R6	Single-vehicle behavior is unchanged when fleet mode is disabled.
R7	Fleet mode aggregates a team score while preserving per-vehicle diagnostics.

C. Design Requirements

The design follows the seven requirements in Table II. They encode two commitments: scoring logic is independent of the participant and cannot be manipulated through privileged feedback; enabling fleet mode must not silently change single-vehicle behavior.

D. Runtime Architecture

The runtime separates scenario construction, vehicle autonomy and race evaluation. A declarative track configuration defines the environment, ordered gates, beacon network, disturbances, participants and scoring rules; after validation, it becomes an arena instance connected to a simulator adapter. The adapter isolates sensing, actuation and time advancement, allowing the controller and referee interfaces to be reused across simulation backends, while the engine-free adapter supports deterministic tests without contributing to the reported results. During execution, the race runner advances the simulator and mediates vehicle-environment interaction. The controller receives an observation containing only participant-local time, allow-listed onboard measurements, physically received beacon packets and optional inter-vehicle messages. It returns a bounded motion command and cannot access simulator state, gate geometry, race progress or referee decisions, so navigation and course progression must be inferred locally. In parallel, the referee uses privileged simulator state to validate crossings and rule violations and to produce event logs, participant results and team-level scores. Referee-derived targets, progress and status are never returned to the controller; disagreements with its local estimate are recorded for offline analysis rather than corrected. Figure 2 summarizes this separation and the controller-visible observation boundary.

IV. VEHICLE, SIMULATOR AND SENSING

This section describes the simulation backend, the vehicle and its sensors, the acoustic and current models, the declarative track configuration and the official observation contract enforced independently of the referee.

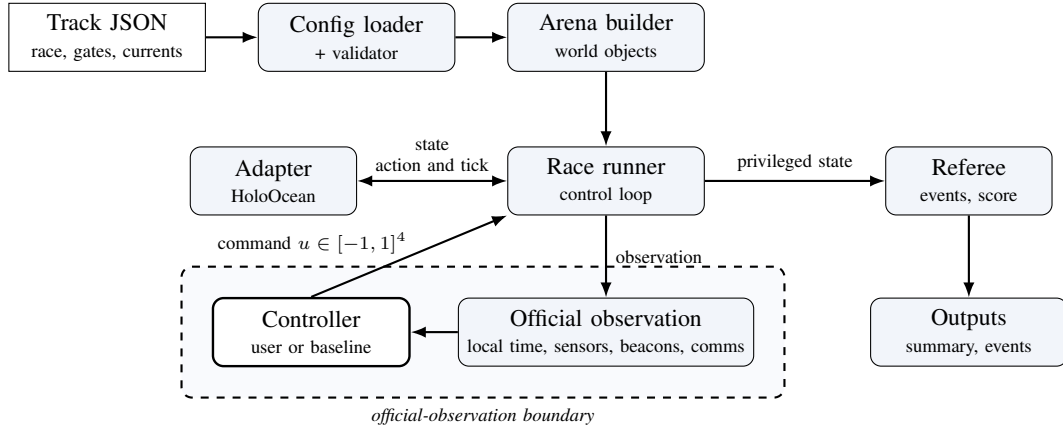


Fig. 2. Runtime architecture and official-observation boundary between the controller and referee.

A. Simulator and Vehicle

The benchmark architecture is not tied to a particular vehicle: adapters provide simulator state, filtered sensing and action mapping to the common race and referee layers. The reference adapter used in every HoloOcean experiment reported here uses HoloOcean [6] with a BlueROV2-class agent [7] in direct eight-thruster mode. The simulator advances at 30 physics ticks per second. The benchmark uses a control timestep of $\Delta t = 0.033$ s, so each control step corresponds to one physics tick and the effective control rate is ≈ 30 Hz.

In the reference HoloOcean adapter, a high-level command $u = [u^{\text{surge}}, u^{\text{sway}}, u^{\text{heave}}, u^{\text{yaw}}]^T$ is mapped to the eight BlueROV2 thrusters by a fixed mixing matrix. The four vertical thrusters receive u^{heave} ; the four horizontal thrusters receive

$$\tau_{1-4} = s_\tau \begin{bmatrix} u^{\text{surge}} + u^{\text{sway}} + \kappa u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} - \kappa u^{\text{yaw}} \\ u^{\text{surge}} + u^{\text{sway}} - \kappa u^{\text{yaw}} \\ u^{\text{surge}} - u^{\text{sway}} + \kappa u^{\text{yaw}} \end{bmatrix}, \quad (5)$$

where $\kappa = 0.35$ is the yaw-mixing gain and s_τ is the thruster scale; each component is clamped to the thruster limit. A controller may instead return the eight thruster values directly. The high-level interface is the default because it is simpler and vehicle-agnostic: a controller expresses intended body-frame motion while the framework handles mixing and depth safety. Direct thruster control remains available when finer actuator authority is required.

B. Sensor Suite

Table III lists the onboard sensors. The official profile combines a forward RGB camera, a depth sensor, an IMU and a DVL; an onboard contact flag is also permitted when configured. Sensors providing world-frame velocity and ground-truth pose, location, rotation and dynamics exist internally for simulation and referee evaluation, but are rejected at the adapter boundary and cannot appear in the official controller observation (Section IV-F).

TABLE III
ONBOARD SENSOR SUITE OF THE BLUEROV2.

Sensor	Rate	Output and configuration
Depth	30 Hz	Scalar depth (m); noise $\sigma = 0$.
IMU	30 Hz	Linear acceleration and angular rate, with bias terms.
DVL	15 Hz	Body-frame velocity (m/s); beam elevation 22.5° , max range 50 m.
Front camera	30 Hz	RGB image, 640×480 , 90° field of view.
Collision	optional	Boolean onboard contact flag when configured.
World-frame velocity, pose-derived heading, pose-derived depth and environmental-current telemetry are not controller-visible in official mode. Pose and velocity sensors may exist internally for simulation and referee evaluation only.		

C. Acoustic Beacon Model

Let $(\hat{n}, \hat{r}_g, \hat{s}_g)$ be the right-handed gate frame of (2), with $\hat{n}$ the passage normal and $\hat{s}_g$ the gate-up axis. Each gate carries one beacon, placed at $b_g = c_g + \delta_n \hat{n} + \delta_r \hat{r}_g + \delta_s \hat{s}_g$ relative to the gate center c_g (default offset $(0, 0, 0.35)$ m along $\hat{s}_g$). For a receiver at position p_r with heading ψ , let $\delta = b_g - p_r$, let $\rho = \|\delta\|$ be the geometric range and let $\tilde{\rho}$ be its noisy delivered measurement. The beacon reports $\tilde{\rho}$, a yaw-relative bearing, an elevation and a signal strength:

$$\rho = \|\delta\|, \quad \tilde{\rho} = \max(0, \rho + \eta_\rho), \quad \eta_\rho \sim \mathcal{N}(0, \sigma_\rho^2), \quad (6)$$

$$\beta = \text{wrap}(\text{atan2}(\delta_y, \delta_x) - \psi) + \eta_\beta, \quad (7)$$

$$\varepsilon = \text{atan2}(\delta_z, \sqrt{\delta_x^2 + \delta_y^2}) + \eta_\varepsilon, \quad (8)$$

$$\text{SS} = \max\left(0, 1 - \frac{\tilde{\rho}}{\rho_{\max}}\right),$$

where $\rho_{\max}$ is the beacon range and $\text{wrap}(\cdot)$ maps to $(-180^\circ, 180^\circ]$. Bearing and elevation carry independent angular perturbations $\eta_\beta, \eta_\varepsilon \sim \mathcal{N}(0, \sigma_\theta^2)$ with σ_θ in degrees and the range carries a single perturbation η_ρ with σ_ρ in metres; each is drawn once per packet and the signal strength derives from the single noisy range $\tilde{\rho}$ so no cleaner distance leaks

as a side channel. A transmission is not received beyond $\rho_{\max}$ or when a Bernoulli dropout fires. The official tracks set σ_θ and σ_ρ to a common per-track value in $\{0.2, 0.45, 0.6\}$ (degrees and metres, respectively) and dropout probabilities up to 0.04. Every ordered gate has one independent periodic transmitter with a unique contiguous identifier B01, ..., BM. All beacons transmit regardless of referee state; at each update, a vehicle receives every in-range, non-dropped packet. Packet randomness is seeded by the run seed, transmitter, receiver and transmission index, making reception independent of participant count and call order.

D. Current Field Model

The current field is a sum of analytic primitives evaluated at the vehicle position p and time t ,

$$\mathbf{v}_c(p, t) = \sum_j \mathbf{v}_j(p, t), \quad (9)$$

where each primitive is one of the following. A *constant* term contributes a fixed velocity $\mathbf{v}_0$. A *localized jet* centered at c_j with radius R_j contributes, for $\|p - c_j\| \leq R_j$,

$$\mathbf{v}_j = \mathbf{v}_0 \omega(d_j), \quad \omega(s) = \begin{cases} \max(0, 1 - \frac{s}{R_j}) & \text{linear,} \\ \exp(-\frac{s^2}{2\sigma_j^2}) & \text{Gaussian,} \end{cases} \quad (10)$$

where $d_j = \|p - c_j\|$ is the jet distance and $\sigma_j = \max(R_j/2, 10^{-6})$ is the jet width; the contribution is zero outside R_j . A *sinusoidal* term modulates one axis, $v_{j,\text{axis}}(t) = v_{0,\text{axis}} + A \sin(2\pi f t + \phi)$. A *vortex* of radius R_j , tangential speed V_t and vertical speed V_z contributes, with $r = \sqrt{\Delta x^2 + \Delta y^2}$ and tangent $\hat{\theta} = (-\Delta y, \Delta x)/r$,

$$\mathbf{v}_j = (\hat{\theta}_x V_t \omega(r), \hat{\theta}_y V_t \omega(r), V_z \omega(r)). \quad (11)$$

The provided *strong* profile superposes a constant set-flow of $[0.75, 1.05, 0]$ m/s, two Gaussian jets, a vortex and a vertical sinusoid; the *medium* profile scales these magnitudes by 0.5. Both named profiles are defined relative to the gate layout and are available on each official circuit. The HoloOcean adapter evaluates the field at every vehicle before each control step, corresponding to one physics tick in the reported setup.

E. Declarative Track Configuration

A track is a self-contained JSON file. Its principal sections are `race` (name, timing mode, laps and duration), `benchmark_task`, `world` (bounds and environment), `start` and `finish`, `track` and `gates` (the ordered gate sequence with centers, passage directions and aperture sizes), `beacon`, `currents` and `current_profiles`, `obstacle_generation` and `obstacles`, `participants` (vehicle, spawn, sensors, controller and optional `start_delay_s`) and `referee` (gate validation, penalties and scoring). The named `benchmark_task` (`clean_gate`, `obstacle_gate`, `current_gate` or `multi_rov`) selects a validation contract. Listing 1 shows a representative excerpt.

Listing 1. Representative excerpt of a track configuration.

```
"track": {
  "gate_inner_size_m": [1.5, 1.5],
  "gate_sequence": ["G01", "G02", "..."]
},
"gates": [
  { "id": "G01", "position": [4.0, -2.0, -3.5],
    "rotation_rpy_deg": [0.0, 0.0, 0.0],
    "passage_direction": [1.0, 0.0, 0.0] }
],
"beacon": {
  "range_m": 90.0,
  "angular_noise_std_deg": 0.2,
  "range_noise_std_m": 0.2,
  "dropout_probability": 0.0,
  "update_rate_hz": 10.0
},
"participants": [
  { "id": "bluerov2_01", "vehicle": "BlueROV2",
    "controller": "rule_gate_baseline",
    "start_delay_s": 0.0 }
],
"referee": {
  "penalties": { "minor_collision_s": 5.0 },
  "gate_validation": {
    "vehicle_clearance_margin_m": 0.1 }
}
```

The three official tracks are summarized in Table IV; all use $1.5 \text{ m} \times 1.5 \text{ m}$ apertures (requirement R4).

The `obstacle_gate` task accepts explicit fixed obstacles or deterministic random boxes generated between consecutive gates with a configured density and minimum clearance. The HoloOcean adapter records the requested and physically spawned counts and an obstacle construction check is accepted only when they match. This verifies that obstacles can be instantiated on a circuit; it is deliberately separate from any claim that a reference controller avoids them.

F. Official Observation Contract

The controller step function receives a dictionary with the fields in Table V. The builder is structurally independent of the referee: its signature contains the track configuration, arena, adapter and participant state, but no referee object. It constructs participant-local elapsed time, requests only allowed onboard sensors from the adapter and queries the beacon field for physically received periodic transmissions. Numeric arrays are made JSON-safe while image arrays are preserved. A controller must infer disturbance effects from its onboard sensors, particularly body-frame DVL measurements. When the optional inter-vehicle acoustic channel is enabled (Section VII), the observation additionally carries a `comms` inbox whose payloads are authored from another controller's own legal information.

V. CONTROLLER INTERFACE AND RULE BASELINE

A central usability goal of Marine Race Arena is to integrate a custom autonomy stack through a small interface without modifying the runner, referee or adapter. This section specifies that interface, the dynamic loading mechanism, the action mapping and the two camera-assisted rule controllers used in the reported single-vehicle comparison.

TABLE IV
OFFICIAL TRACK CONFIGURATIONS.

Track file	Track name	Gates	Laps	Total gates	Length (m)	Intended use
marine_race_horseshoe_bay.json	Marine Race Horseshoe Bay	12	1	12	93.8	Clean-gate horseshoe baseline
marine_race_vertical_serpent.json	Marine Race Vertical Serpent	17	1	17	118.3	Vertical and slalom gate sequencing
marine_race_mixed_endurance.json	Marine Race Mixed Endurance	22	1	22	206.3	Longer endurance and current-oriented layout

TABLE V
TOP-LEVEL FIELDS OF THE OFFICIAL OBSERVATION.

Field	Meaning
local_time_s	Elapsed time since this vehicle's release; it is not official race time.
sensors	Filtered onboard dictionary containing only configured camera, depth, IMU, DVL and optional collision measurements.
beacons	List of physically received packets from independent transmitters; it may be empty or contain several ids.
comms	Present only when the optional inter-vehicle acoustic channel is enabled: an inbox of messages received from teammates, each with the sender id, the controller-authored payload and a receiver-local reception timestamp (Section VII).

A. Controller Interface

A controller implements three methods: `reset(mission_info)`, called once before the run with a minimal static assignment; `step(observation)`, which receives the official observation of (3) at every control tick and returns the command of (4); and `close()`, called at teardown. The contract is duck-typed: any object exposing the three callables is accepted, so a participant need not depend on framework base classes. A manual controller may raise a dedicated stop exception from `step` to end a run cleanly; other exceptions are recorded as controller errors. Listing 2 shows a minimal example of local progression.

Listing 2. Minimal custom controller.

```
class MyController:
    def reset(self, mission_info):
        self.tracker = LocalCourseTracker(
            initial_beacon_id=mission_info[
                "initial_beacon_id"],
            total_beacons=mission_info[
                "total_beacons"],
            laps=mission_info["laps"])

    def step(self, observation):
        sensors = observation["sensors"]
        state = self.tracker.update(
            local_time_s=observation["local_time_s"],
            beacons=observation["beacons"],
            camera_image=sensors.get("FrontCamera"),
            dvl_velocity=sensors.get("DVLsensor"))
        if state.finished:
            return {"surge": 0.0, "sway": 0.0,
                    "heave": 0.0, "yaw": 0.0}
```

TABLE VI
HIGH-LEVEL CONTROLLER COMMAND FIELDS OF (4).

Field	Interpretation
surge	Longitudinal body-frame thrust; positive moves forward.
sway	Lateral body-frame thrust; sign follows the adapter mixing of (5).
heave	Vertical thrust; additionally constrained by depth-safety logic near the bounds.
yaw	Yaw-rate command; sign follows the adapter mixing of (5).

```
# ... map state.beacon and camera to a command ...
return {"surge": 0.3, "sway": 0.0,
        "heave": 0.0, "yaw": 0.0}

def close(self):
    pass
```

The high-level command fields are listed in Table VI; values are normalized to $[-1, 1]$ and clamped by the framework before the action mapping. In fleet mode a controller may additionally return a small message payload alongside its command and read incoming teammate messages from the observation, using the optional inter-vehicle acoustic channel of Section VII.

B. Dynamic Loading

A controller is selected at run time in one of three ways: a built-in alias such as `rule_gate_baseline` or `rule_gate_center_then_commit`; a dotted module path with an explicit class; or a file path together with `--controller-class`, which is imported dynamically. For inspection and data collection, the runner can also be launched with the manual keyboard or pygame controllers. This lets an external policy be evaluated without modifying the package:

```
python -m marine_race_arena.scripts.run_marine_race \
--track ../marine_race_horseshoe_bay.json \
--benchmark-task clean_gate \
--controller path/to/my_controller.py \
--controller-class MyController \
--adapter holoocean --official --headless
```

C. Action Mapping

The framework clamps the returned command to $[-1, 1]^4$ and applies a depth-safety rule that suppresses heave that would drive the vehicle past $z_{\min}$ or $z_{\max}$. The selected adapter then maps this portable body-frame command to its vehicle; the reference HoloOcean adapter uses (5). That adapter may alternatively receive raw eight-thruster values, while the high-level interface remains the vehicle-independent default.

D. Controller-Local Course Progression

Both official rule controllers own a `LocalCourseTracker`; the runner supplies no progression signal. The tracker initializes from the public mission assignment `initial_beacon_id` and maintains a local expected beacon, completed count, lap and status through

SEARCH $\rightarrow$ APPROACH $\rightarrow$ VISUAL_ALIGN
 $\rightarrow$ COMMIT $\rightarrow$ VERIFY_EXIT $\rightarrow$ ADVANCE,

ending in FINISHED after the final locally confirmed beacon. It filters the received list for its own expected id; packets from other transmitters are not labeled or selected by the framework. Temporary visual loss, a missing packet, an isolated range jump, a range rise without a prior close approach and a commit without forward DVL displacement cannot advance the state.

The tracker enters and completes a passage only when persistent camera, acoustic and DVL evidence agrees with the expected beacon. Incomplete or inconsistent evidence returns it to local acquisition without advancing. The two reference controllers and their local-progression logic were designed and validated for the gate geometry, beacon placement, sensing configuration and vehicle dynamics of the three official circuits used in this study. After each non-final local advancement, the tracker holds an exit-clearance phase; after the final advancement, it enters FINISHED directly and commands zero. Controller-local snapshots and independent referee crossings are logged separately for offline comparison and never fed back into control.

E. Rule-Based Baselines

The two official reference controllers are deterministic and interpretable rule-based baselines rather than optimized policies. They use the same onboard inputs: received acoustic beacon packets, the forward RGB camera, depth and IMU measurements and body-frame DVL velocity. Both combine acoustic guidance, visual centering, depth hold and DVL-based passage verification through the shared `LOCALCOURSETRACKER` shown in Fig. 3.

Continuous Servo steers toward the expected beacon and uses the camera to refine alignment as the gate becomes visible. Depth feedback maintains the vertical reference, while DVL motion provides evidence that the vehicle has crossed and cleared the gate. Its defining behavior appears during COMMIT and VERIFY_EXIT: the controller continues applying bounded visual corrections while moving through the aperture. It therefore keeps servoing toward the observed image center throughout the passage.

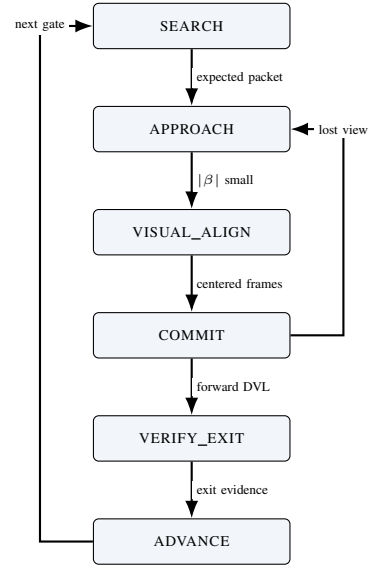


Fig. 3. The `LOCALCOURSETRACKER` state machine shared by both rule controllers. ADVANCE loops to SEARCH for the next gate and reaches FINISHED after the last confirmed passage.

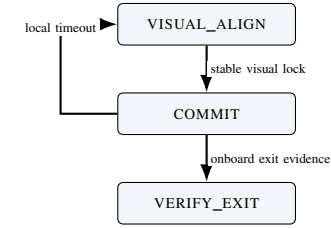


Fig. 4. Center-then-commit passage in the COMMIT and VERIFY_EXIT states, with controller-local confirmation and recovery.

F. A Center-Then-Commit Rule Controller

Center-then-Commit retains the same observations, acoustic and visual guidance, shared tracker and local confirmation criterion as *Continuous Servo*. The only difference is the gate-passage strategy. After establishing a stable visual-acoustic lock, it stops chasing subsequent image-centroid changes during COMMIT and VERIFY_EXIT. It instead advances through the aperture while maintaining the locked depth and attitude from onboard depth and IMU feedback; the same DVL-based evidence is used to verify the passage.

Because the observations, `LOCALCOURSETRACKER` and confirmation logic remain unchanged, the matched comparison of Section VIII-E isolates the effect of continuous visual correction versus a stabilized committed passage.

VI. REFEREE, GATE VALIDATION AND SCORING

The referee evaluates progress independently of the controller (requirement R3) using privileged ground-truth state. This section formalizes the gate-crossing test, the referee state machine and its arbitration order, the penalty model and the single-vehicle score.

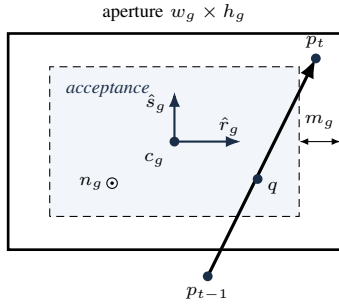


Fig. 5. Gate-crossing geometry and the safety-margin-adjusted acceptance region.

A. Gate Frame and Crossing

The referee reuses the right-handed gate frame $(\hat{n}, \hat{r}_g, \hat{s}_g)$ of (2), defined in Section III from the passage direction n_g (with the vertical-normal fallback to the world $\hat{x}$ axis). For consecutive referee-side positions p_{t-1}, p_t , the signed distances to the gate plane are

$$d_{t-1} = \hat{n}^\top (p_{t-1} - c_g), \quad d_t = \hat{n}^\top (p_t - c_g). \quad (12)$$

A candidate crossing requires a sign change in the *correct direction*,

$$d_{t-1} \leq 0 \leq d_t \wedge (p_t - p_{t-1})^\top \hat{n} > 0, \quad (13)$$

i.e. travel from the negative to the positive side along the normal. The intersection of the segment with the plane is

$$\lambda = \frac{-d_{t-1}}{d_t - d_{t-1}}, \quad q = p_{t-1} + \lambda(p_t - p_{t-1}), \quad \lambda \in [0, 1], \quad (14)$$

and the crossing is accepted only when q lies inside the aperture, shrunk by the configured safety margin m_g ,

$$|\hat{r}_g^\top (q - c_g)| \leq \frac{w_g}{2} - m_g, \quad |\hat{s}_g^\top (q - c_g)| \leq \frac{h_g}{2} - m_g. \quad (15)$$

Gates must be passed *in sequence*: the referee tests only the expected gate g_ℓ , advancing the index $\ell \leftarrow \ell + 1$ on a valid crossing and incrementing the lap when the sequence completes. A crossing of a *non-target* gate aperture is a missed-gate event; a sign change in the wrong direction is logged but neither penalized nor counted as progress. Fig. 5 illustrates the test and Fig. 6 the principal automatic lifecycle.

B. Referee State Machine and Arbitration

Each participant occupies one of the states NOT_STARTED, RUNNING, FINISHED, DNF, TIMEOUT, STUCK, DSQ, MANUAL_STOP or CONTROLLER_ERROR; all but the first two are terminal. A participant is released to RUNNING when the race clock reaches its start delay (Section VII). On each tick of a running participant, events are evaluated in a fixed priority order that defines the arbitration: (1) a controller error transitions to CONTROLLER_ERROR and returns; (2) out-of-bounds; (3) generic collision; (4) obstacle collision; (5) stuck accumulation; (6) duration timeout; (7) gate progression. Steps (2) to (5) accrue penalties without terminating; only (1), (6),

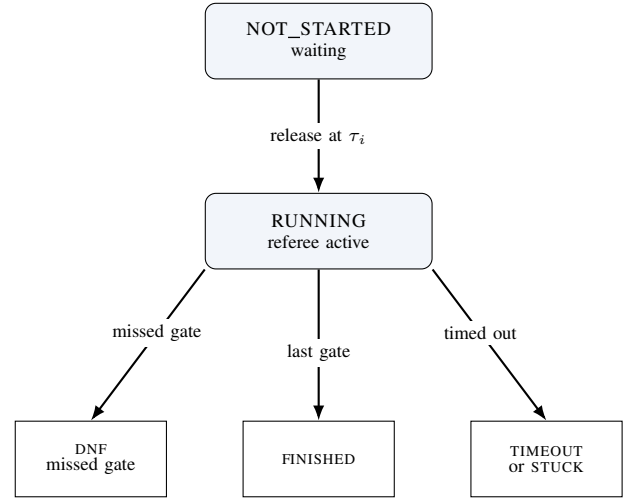


Fig. 6. Simplified automatic participant lifecycle; manual-stop and controller-error terminals are omitted.

TABLE VII
DEFAULT REFEREE EVENTS, PENALTIES AND TERMINAL CONDITIONS.

Event	Penalty	Terminal
Gate, bar or generic collision	5.0 s each	no
Obstacle collision	5.0 s each [†]	no
Out of bounds	10.0 s each	no
Stuck (soft)	15.0 s per episode	no
Missed gate	N/A	yes (DNF)
Timeout	N/A	yes
Gate-timeout stuck	N/A	yes
Inter-vehicle proximity	0 or 5.0 s per pair	no (team-level)

[†]Per-obstacle value if specified, else the collision default. Collision, obstacle, out-of-bounds and inter-vehicle proximity charges use independent configured cooldowns (default 1.0 s); stuck uses one latch per episode and terminal events have no cooldown.

a missed gate in (7) or an external gate-timeout produce a terminal state. To avoid double counting, an obstacle-collision tick suppresses the generic collision flag. Repeated collision, obstacle and out-of-bounds charges use independent cooldowns; inter-vehicle proximity uses its own team-level cooldown, stuck is latched per episode and terminal events are not cooldown-limited.

C. Penalties and Termination

Penalties accumulate into a per-participant total P_i ; the applied defaults are listed in Table VII. The non-trivial conditions are formalized as follows. The instantaneous speed is $s_t = \|p_t - p_{t-1}\|/\Delta t$; a stuck accumulator integrates low-speed time,

$$T^{\text{stuck}} \leftarrow \begin{cases} T^{\text{stuck}} + \Delta t, & s_t < v_{\text{stuck}} \\ 0, & \text{otherwise,} \end{cases} \quad (16)$$

and a soft penalty is charged once per continuous low-speed episode when $T^{\text{stuck}} \geq T_{\text{max}}^{\text{stuck}}$, with $v_{\text{stuck}} = 0.02$ m/s.

The configured $T_{\max}^{\text{stuck}}$ is 45 s on Horseshoe Bay and 65 s on Vertical Serpent and Mixed Endurance. An out-of-bounds event is $p_t \notin \mathcal{B}$. A duration timeout, disabled by default, fires when $t - t_{\text{green}} > T_{\max}$. A missed gate sets DNF when `missed_gate_dnf` is enabled (the default), but carries no time penalty, so a disqualifying error is never cheaper than a slow-but-legal lap.

D. Single-Vehicle Score and Ranking

For a finished participant the official time T_i^{official} is measured between the first and last gate (or green-to-finish, per the timing mode) and the score is the penalized time

$$T_i^{\text{pen}} = T_i^{\text{official}} + P_i. \quad (17)$$

Ranking places finished participants before unfinished ones and is defined by the ascending lexicographic key

$$\kappa_i = \begin{cases} (0, T_i^{\text{pen}}, 0, 0, 0, \text{id}_i), & f_i = 1, \\ (1, -G_i^{\text{done}}, n_i^{\text{coll}}, P_i, \text{dist}_i, \text{id}_i), & f_i = 0, \end{cases} \quad (18)$$

where $f_i = 1$ denotes a finished vehicle, G_i^{done} is the number of completed gates, n_i^{coll} the collision count and dist_i the distance to the next gate; the participant id is the deterministic final tie-break. Thus, finished vehicles are ordered by penalized time. Unfinished vehicles are ordered by progress, then by fewer collisions, fewer penalties and proximity to the next gate. This ordering preserves the information in partial runs rather than discarding them.

E. Event Logging

Every run emits a line-delimited event log and a machine-readable summary (requirement R5). Events include `gate_passed`, `wrong_direction`, `collision`, `obstacle_collision`, `out_of_bounds`, `stuck`, `race_finish` and `dnf`, each timestamped and attributed to a participant. For each participant, the summary records the status, official and penalized times, completed gates, collision count, out-of-bounds events, missed gates and stuck events, together with the final ranking. In fleet mode, it also carries the team summary of Section VII.

VII. FLEET AND TEAM EVALUATION

Fleet mode runs several vehicles as a single cooperative team through the same circuit, so the benchmark can measure how quickly the team completes the course, rather than pitting them as competitors against each other (requirement R7). The benchmark models one team only: all vehicles share a single simulated world, each controller keeps an independent local course estimate and the referee separately keeps one scoring state per vehicle. Enabling fleet mode does not alter single-vehicle behavior (R6).

A. Staggered Release and Spawn Geometry

Each participant i is assigned a release delay $\tau_i = (i - 1)\Delta\tau$ for a start gap $\Delta\tau$, so vehicles enter the course at $0, \Delta\tau, 2\Delta\tau, \dots$. A waiting vehicle receives a zero command, its controller `step` is *not* called and neither stuck nor gate-timeout timers accumulate. At release, the referee logs a `participant_released` event and resets the vehicle's progress timers, so a delayed start is never misclassified as stuck or timed out. Spawn poses are laterally separated about the base spawn to reduce launch interference. Using zero-based $k = 0, \dots, N - 1$, the k -th vehicle is offset by

$$o_k = (-1)^k \left\lceil \frac{k}{2} \right\rceil s \quad (19)$$

along the right axis $(-\sin\psi_0, \cos\psi_0)$ of the base yaw ψ_0 , giving the alternating pattern $0, -s, +s, -2s, +2s, \dots$ for spacing s ; a candidate that would fall outside the world bounds $\mathcal{B}$ is scaled inward until admissible.

B. Inter-Vehicle Proximity Diagnostics

Inter-vehicle proximity events are detected on the referee side and are never exposed to controllers. For two running vehicles a, b the detector applies a separable cylinder test

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2} \leq r_{xy} \wedge |z_a - z_b| \leq r_z, \quad (20)$$

with hysteresis (a pair is released only when the horizontal distance exceeds a release threshold r_{rel} or the vertical distance exceeds r_z) and a refractory cooldown to debounce sustained proximity.

C. Inter-Vehicle Acoustic Communication

Because the fleet is a single cooperative team, Marine Race Arena provides an optional channel so that vehicles can share information while running the course. It is off by default and changes nothing for single-vehicle runs (R6). A controller transmits by returning a small message payload alongside its command and receives an `inbox` of teammates' messages in its observation; what a message contains and how it is used are entirely the user's responsibility, so the framework provides only the transport.

To stay faithful to the medium rather than model a perfect radio link, the channel reproduces the constraints of the underwater acoustic channel of Section II. A message broadcast by vehicle i reaches a teammate j only if the two are within a maximum range $R_{\max}$, after a range-dependent delay

$$\tau_{ij} = \frac{d_{ij}}{c} + \tau_{\text{proc}}, \quad (21)$$

where d_{ij} is the inter-vehicle distance, c the speed of sound and τ_{proc} a fixed processing delay. Each delivery is also dropped with probability p_{loss} . Payloads are capped to a small size to model the low bandwidth and a vehicle may not transmit faster than a configured per-sender interval. Packet loss is drawn from a seeded generator, so a run remains reproducible.

This implemented acoustic-inspired transport is not a fully characterized channel model. It provides the substrate for the

leader-follower coordination of Section VII-D and is exercised as part of that coordination validation. Its parameters (packet loss, maximum range, latency and freshness window) are not independently swept.

D. Leader-Follower Coordination

A staggered team of identical controllers remains separated in time, whereas a heterogeneous team does not. A faster follower released behind a slower leader catches up, causing the vehicles to bunch at gates where they converge on the same aperture center. We therefore provide a coordination controller that uses only legal information to maintain a spaced convoy. It wraps, rather than replaces, a gate-passing controller that exposes a `LocalCourseTracker`, keeping coordination and gate navigation separate. Both official camera-assisted controllers satisfy this interface.

The controller is a thin, fully distributed layer over the acoustic channel of (21). Each vehicle is assigned a predecessor from the static release order in its legal mission information; the frontmost vehicle has no predecessor and races freely. Within the channel's transmit-rate limit, every vehicle periodically broadcasts exactly the three fields `local_beacon_index`, `local_lap` and `local_status`, read from its own base controller's `LocalCourseTracker`. The receiver treats these fields as estimates, retains the most recently received predecessor report and ignores it after a 2.5s freshness window. Let $\hat{g}_i$ denote cumulative locally estimated progress reconstructed from the reported lap and beacon index. A follower yields while its predecessor is fewer than Δg locally estimated gates ahead,

$$\text{yield}_j \iff \hat{g}_{\text{pred}(j)} - \hat{g}_j < \Delta g, \quad (22)$$

and otherwise it runs its wrapped base controller unchanged. While yielding, the wrapper still updates the base tracker from onboard observations but commands zero motion. This deliberate wait can therefore trigger the independent stuck penalty and remains visible in the result. A predecessor locally reported as finished, a missing report or a report older than the freshness window stops blocking. With the channel disabled no report is delivered and the wrapper reduces to the uncoordinated base behavior.

The policy reads only the official observation, its own tracker, the static predecessor assignment and controller-authored messages; referee progress can disagree with either local estimate but never changes a coordination decision.

E. Team-Level Score

Only the `team_summary` is the official fleet result; per-vehicle rows are diagnostics. For a team $\mathcal{P}$ of N vehicles with G_{exp} expected gates each,

$$G_{\text{team}}^{\text{exp}} = N G_{\text{exp}}, \quad G_{\text{team}}^{\text{done}} = \sum_{i \in \mathcal{P}} G_i^{\text{done}}. \quad (23)$$

The team finishes only if every vehicle finishes,

$$\text{finished}_{\text{team}} = \bigwedge_{i \in \mathcal{P}} \text{finished}_i, \quad (24)$$

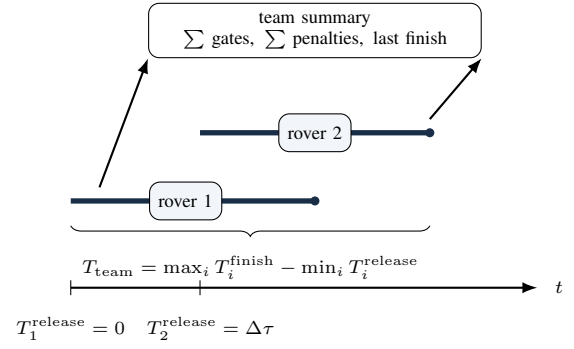


Fig. 7. Team-level scoring for a staggered two-rover fleet.

in which case the team elapsed time spans the first release to the last finish,

$$T_{\text{team}} = \max_i T_i^{\text{finish}} - \min_i T_i^{\text{release}}, \quad (25)$$

and the penalized team score aggregates every per-vehicle penalty together with the team-level inter-vehicle term,

$$T_{\text{team}}^{\text{pen}} = T_{\text{team}} + \sum_{i \in \mathcal{P}} P_i + P_{\text{ivc}}, \quad (26)$$

where P_i is the per-vehicle penalty total of (17) and P_{ivc} counts each inter-vehicle proximity event once per pair, not once per vehicle. Fig. 7 illustrates the aggregation.

VIII. EXPERIMENTAL EVALUATION

This evaluation does not seek an optimal underwater racing controller. It tests whether the benchmark, official observation contract, referee and team scoring behave as specified, compares the two reference controllers across every reported condition and reports where the reference baseline fails.

A. Protocol and Metrics

The evaluation comprises 78 real-HoloOcean runs generated under a frozen source fingerprint recorded in the artifact manifest (Section VIII-G). It covers both reference controllers on the three clean tracks and under the `medium` and `strong` Horseshoe Bay currents, homogeneous two-vehicle fleets and matched three-vehicle coordination trials. Clean, current and fleet cases use five seeds, while coordination uses three seeds under simultaneous and staggered release. Figure 8 shows the official track layouts.

All performance trials use the reference HoloOcean adapter, a BlueROV2-class vehicle, official onboard-only observations, no current compensation or obstacles and the standard gate apertures. Separate setup checks confirm that both named current profiles and seeded static obstacles can be instantiated on every circuit; these are construction checks rather than obstacle-avoidance trials.

The reported metrics are completion, completed gates, official and penalized time, collisions, out-of-bounds events and stuck events. Fleet and coordination trials also report inter-vehicle proximity and team times. Time statistics use finished runs only, whereas gate and event means use all seeds;

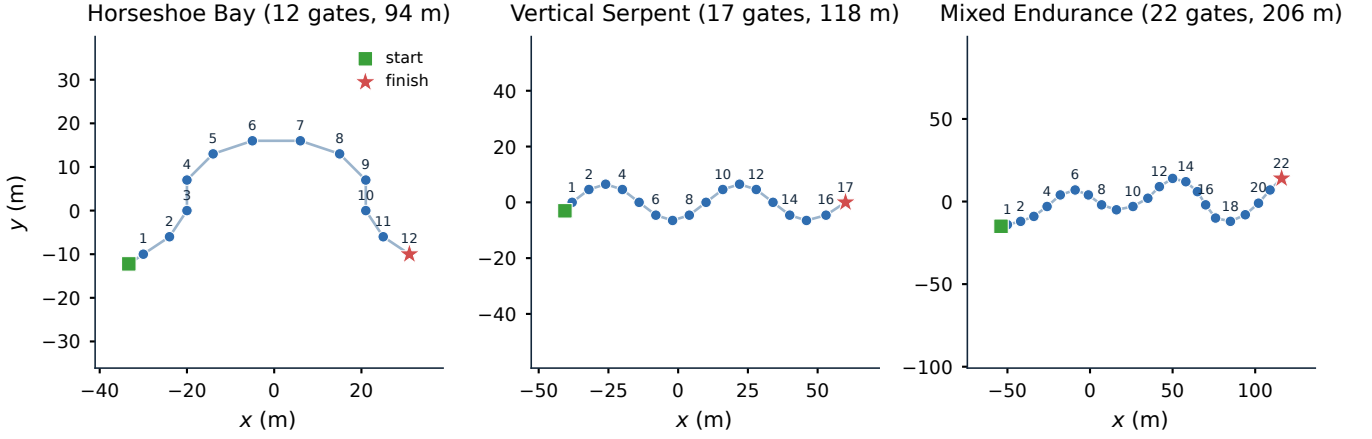


Fig. 8. Top-down (x, y) layouts of the three official tracks (released JSON): ordered gates, start (square) and finish (star).

dispersions are sample standard deviations. Controller-local progression is audited against referee progression. False or missed advancements and inconsistencies in final progress or finish state are retained as audit failures.

B. Single-Vehicle Clean Tracks

Table VIII reports the clean-track results over five seeds. Continuous Servo finishes every Horseshoe Bay and Vertical Serpent run but only 2/5 Mixed Endurance runs. Center-then-Commit finishes all Horseshoe Bay and Mixed Endurance runs and 4/5 Vertical Serpent runs. The mean completed-gate counts are respectively 12.0, 17.0 and 17.0 for Continuous Servo and 12.0, 14.8 and 22.0 for Center-then-Commit. No clean run records a gate or world collision, out-of-bounds event or stuck event.

C. Staggered Fleet and Team Scoring

The bottom block of Table VIII reports homogeneous two-vehicle fleet validation on Horseshoe Bay over five seeds. With either controller, every team finishes 24/24 gates without gate or world collisions, inter-vehicle proximity events, out-of-bounds events or stuck events. Mean team elapsed and penalized times are both 303.1 ± 3.2 s for Continuous Servo and 308.0 ± 5.8 s for Center-then-Commit. The 90 s release gap deliberately produces a low-contact case that validates multi-vehicle spawning, staggered release, independent per-vehicle referee state, ranking and team aggregation for both official controllers; the interacting case is studied with three vehicles below.

D. Behaviour Under Currents

Performance degrades as current strength increases. Under the Horseshoe Bay medium profile, Continuous Servo finishes 2/5 runs, averages 8.4/12 completed gates and 43.4 gate and world collisions per run; its finished-only official and penalized times are 337.9 ± 19.2 s and 692.9 ± 75.7 s. Center-then-Commit finishes 3/5 runs, averages 8.8/12 gates and 25.2 gate and world collisions, with finished-only times of 217.3 ± 1.5 s and 223.9 ± 1.8 s. Neither controller finishes a strong run:

Continuous Servo averages 3.0/12 gates and no gate or world collision, while Center-then-Commit averages 3.2/12 gates and 0.8 gate and world collisions. Metadata confirms physical current coupling in every run.

Two features of these numbers deserve comment. First, the collision counts are *non-monotonic* in current strength: the medium profile records many gate and world collisions (43.4 and 25.2) while the strong profile records almost none (0.0 and 0.8). This is not evidence of better control under the stronger current. Under the strong profile, the vehicles complete only between 3.0 and 3.2 of 12 gates and stall early, so they rarely reach the gate-dense regions where contacts accumulate; fewer collisions here reflect less progress, not more skill. Second, the strong profile is best read as a saturation regime. Its constant set-flow component alone is $[0.75, 1.05, 0]$ m/s, a magnitude of 1.29 m/s, about $2.7\times$ the mean course speed the same vehicle sustains on clean Horseshoe Bay (93.8 m in 194.5 s, i.e. ≈ 0.48 m/s). The strong profile therefore substantially limits forward course progress and functions as a stress case, whereas the medium profile (0.65 m/s constant component) is the more informative disturbance regime. The quantitative current sweep is limited to Horseshoe Bay (Section IX-A).

E. Reference-Controller Comparison

The two camera-assisted controllers of Section V offer different trade-offs between speed and robustness, with neither dominating the other; Fig. 9 summarizes their completion across every reported condition. On the short, planar Horseshoe Bay circuit, they are effectively tied. Both finish all 12 gates in every seed with no gate or world collision, out-of-bounds event or stuck event. Their mean official and penalized times are 194.5 ± 4.8 s for Continuous Servo and 197.7 ± 7.1 s for Center-then-Commit, so the staged controller is 3.2 s (1.7%) slower in this matched case with no measured collision advantage.

The two controllers separate on the harder tracks and under disturbance. On Vertical Serpent, Continuous Servo is more reliable (5/5 versus 4/5 finishes). On the long Mixed

TABLE VIII
FINAL HOLOOCEAN CLEAN-TRACK, CURRENT AND HOMOGENEOUS TWO-VEHICLE FLEET RESULTS OVER FIVE SEEDS PER CASE. TIMES ARE FINISHED-RUNS-ONLY; GATE, COLLISION AND EVENT MEANS ARE OVER ALL FIVE SEEDS.

Setting	Controller and track	Seeds	Status	Gates $\uparrow$	Time (s) $\downarrow$	Collisions $\downarrow$
Clean	Continuous servo, Horseshoe Bay	5	5/5 FIN	12.0/12	194.5 ± 4.8	0.0
	Center-then-commit, Horseshoe Bay	5	5/5 FIN	12.0/12	197.7 ± 7.1	0.0
	Continuous servo, Vertical Serpent	5	5/5 FIN	17.0/17	236.9 ± 6.4	0.0
	Center-then-commit, Vertical Serpent	5	4/5 FIN	14.8/17	241.1 ± 1.8	0.0
	Continuous servo, Mixed Endurance	5	2/5 FIN	17.0/22	394.7 ± 3.9	0.0
	Center-then-commit, Mixed Endurance	5	5/5 FIN	22.0/22	410.7 ± 8.8	0.0
Currents	Continuous servo, Horseshoe medium	5	2/5 FIN	8.4/12	337.9 ± 19.2	43.4
	Center-then-commit, Horseshoe medium	5	3/5 FIN	8.8/12	217.3 ± 1.5	25.2
	Continuous servo, Horseshoe strong	5	0/5 FIN	3.0/12	—	0.0
	Center-then-commit, Horseshoe strong	5	0/5 FIN	3.2/12	—	0.8
Fleet, gap 90 s	Continuous servo, team aggregate	5	5/5 FIN	24.0/24	303.1 ± 3.2	0.0
	Center-then-commit, team aggregate	5	5/5 FIN	24.0/24	308.0 ± 5.8	0.0

Times are mean $\pm$ sample standard deviation (ddof= 1) over finished runs; a cell with a single finished run shows the mean without a spread. Gates and collisions are five-seed means; fleet values are team-level and collisions combine gate and world events. Medium-current penalized times are 692.9 ± 75.7 s for continuous servo and 223.9 ± 1.8 s for center-then-commit.

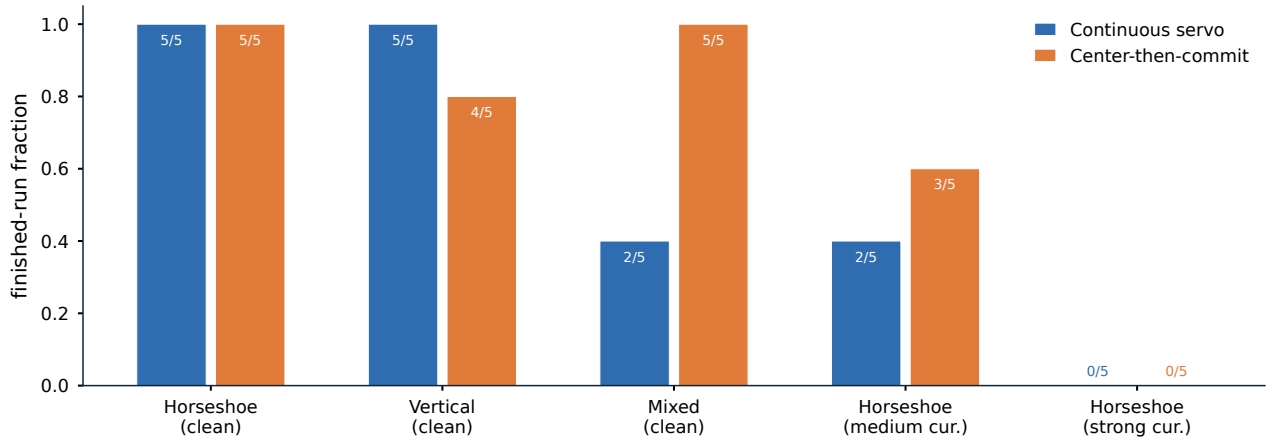


Fig. 9. Finished-run fraction of the two reference controllers over five seeds per condition; each label reports finished runs out of five.

Endurance course, the pattern reverses more clearly: Center-then-Commit finishes all five seeds and completes every gate (22.0/22), whereas Continuous Servo finishes only 2/5 and averages 17.0/22 gates. Under the medium current, the staged controller is again the more robust of the two. It finishes more runs (3/5 versus 2/5), incurs roughly half the collisions (25.2 versus 43.4) and completes its finished runs far faster (217.3 s versus 337.9 s). The committed-passage phase, which stops chasing image-centroid changes once locked, thus trades a small amount of clean-track speed for steadier passages when the course is long or the vehicle is pushed off line; the continuous-servo policy retains a slight edge only where constant re-centering is cheap. The comparison is limited to the three official tracks, two current profiles and the evaluated seeds.

F. Three-Vehicle Coordination

This validation tests local leader-follower coordination under contact dynamics. It uses clean Horseshoe Bay in official

mode with three vehicles, no currents or obstacles, two start gaps (8.0 and 0.0 s) and three seeds. The leader runs Continuous Servo and both followers run Center-then-Commit; every controller advances its own LocalCourseTracker and heartbeats carry only local estimates. No controller or course parameter is retuned between conditions. We report the recommended one-gate margin LF(1), the conservative two-gate margin LF(2) and the matched uncoordinated baseline (Table IX).

Coordination removes physical interaction that the uncoordinated convoy suffers and the recommended LF(1) margin does so at the lowest time cost. At an 8 s gap, the uncoordinated convoy finishes only 2/3 runs, averages 34.3/36 team gates, 127.3 gate and world collisions and 2.0 inter-vehicle proximity events per run; LF(1) and LF(2) both finish 3/3 with no gate, world or proximity events and LF(1) is faster (266.0 ± 8.3 s versus 285.7 ± 3.8 s team time). At simultaneous release, the uncoordinated convoy still finishes 3/3 but averages 9.3 gate and world collisions and 2.3 inter-

TABLE IX
THREE-VEHICLE COORDINATION ON CLEAN HORSESHOE BAY OVER THREE SEEDS PER CONDITION.

Setting	Policy	Seeds	Status	Gates $\uparrow$	Team time (s) $\downarrow$	Events $\downarrow$
Gap 0 s	No coordination	3	3/3 FIN	36.0/36	219.7 ± 2.5	9.3 / 2.3 / 0.0
	LF(1)	3	3/3 FIN	36.0/36	262.3 ± 7.7	0.0 / 0.0 / 0.0
	LF(2)	3	3/3 FIN	36.0/36	288.3 ± 1.3	0.0 / 0.0 / 1.0
Gap 8 s	No coordination	3	2/3 FIN	34.3/36	254.0 ± 24.5	127.3 / 2.0 / 0.0
	LF(1)	3	3/3 FIN	36.0/36	266.0 ± 8.3	0.0 / 0.0 / 0.0
	LF(2)	3	3/3 FIN	36.0/36	285.7 ± 3.8	0.0 / 0.0 / 0.0

Times are raw team elapsed mean $\pm$ sample standard deviation over full-team finishes; gates and events are three-seed means. Events are GW / IV / S: gate and world collisions, inter-vehicle proximity and stuck. No run records an out-of-bounds event. Penalized time differs for no coordination at gaps 0 and 8 s (266.4 ± 64.1 and 564.0 ± 463.0 s) and LF(2) at gap 0 s (303.3 ± 1.3 s).

vehicle proximity events; LF(1) again eliminates both event types at 262.3 ± 7.7 s, while the more conservative LF(2) is slower (288.3 ± 1.3 s raw, 303.3 ± 1.3 s penalized) and, because its longer hold can trip the independent stuck timer, records one stuck event per run. Across both gaps LF(1) finishes every run with zero gate, world, proximity, out-of-bounds and stuck events and is faster than LF(2); LF(2) keeps a larger nominal gate separation but did not improve safety in these conditions, which is why LF(1) is the recommended default and LF(2) is retained only as a conservative comparison. This evidence is limited to one clean track, three vehicles, two gaps and three seeds (Section IX-A).

G. Reproducibility

Every run emits a structured event log and summary. The repository provides the exact commands used here [19] and an artifact manifest that records, for each run, the experiment-generation commit, source-tree fingerprint, HoloOcean environment, track, controller, seed, command, fallback-disabled status and physical current-coupling status. A single configuration-driven entry point and per-track JSON files make each official run reproducible from one command.

IX. DISCUSSION, LIMITATIONS AND FUTURE WORK

The experiments show that completion depends on both course structure and environmental conditions. The two reference controllers clear the short clean circuit but separate on the longer track and under current; success on clean tracks does not imply robustness to disturbances. The coordination study shows that a fully distributed leader-follower layer, using only legal onboard estimates and an acoustic-inspired channel, removes the physical interaction experienced by an uncoordinated heterogeneous convoy, with the one-gate margin achieving this result at the lowest time cost. Throughout, controller-local progression and independent referee scoring remain separate. Their occasional disagreement is logged rather than reconciled, allowing the same contract to score arbitrary controllers.

A. Limitations

The evaluation has several limitations. The reference controllers are rule-based, all results are obtained in HoloOcean,

and no real-vehicle or learning-based validation is included. The acoustic channel is evaluated only within the coordination task, without an independent analysis of packet loss, range or latency. The strong-current profile mainly represents a saturation condition, while the revised beacon-noise model preserves the numerical behavior of the original implementation rather than introducing newly calibrated sensor statistics. These limitations define the present scope and indicate natural directions for extending the released benchmark.

B. Future Work

Four directions follow.

(i) *Cooperative fleet strategies beyond leader-follower conveying.* The leader-follower coordination of Section VII-D is validated on clean Horseshoe Bay (Section VIII-F). Further studies can cover other tracks, currents, obstacles and larger fleets, as well as formation keeping, cooperative overtaking, dynamic role assignment and robustness to uncertain relative localization or heavier acoustic loss. Such strategies must respect the low bandwidth and high latency of acoustic communication [16] described in the AUV-formation literature [17], [18].

(ii) *Competitive multi-team and heterogeneous-vehicle evaluation.* The cooperative fleet setting can be extended to multiple teams or vehicle families with different dynamics and sensing suites competing under the same referee and observation contract. This requires team-isolated scoring, ranking across teams, controlled interaction policies and normalization rules that compare heterogeneous platforms without hiding their physical differences.

(iii) *Current compensation and broader disturbance validation.* The three official circuits already expose the graded medium and strong profiles of (9) to (11); the present quantitative sweep is limited to Horseshoe Bay. Controllers can be evaluated on the remaining circuits while explicitly rejecting these disturbances. One legal direction is a cascaded design with an inner loop that servos DVL-measured body velocity to the guidance setpoint, beneath outer beacon and vision guidance, without access to environmental-current telemetry.

(iv) *Learning-based controllers.* The controller-visible observation and normalized action contract provide the basis for a future Gymnasium-compatible learning interface. Such an

interface requires a fixed observation encoding, an explicit training reward, step-wise environment control, and validation of learned policies. Implementing and evaluating imitation- and reinforcement-learning controllers is left for future work.

X. CONCLUSION

This paper presented Marine Race Arena, a configurable benchmark for autonomous underwater gate racing in HoloOcean. A replaceable adapter separates vehicle and simulator integration from the race model, while an official observation contract keeps controller inputs distinct from the privileged state used by the independent referee. The same evaluation principles support individual vehicles and cooperative fleets with reproducible configuration, scoring and logging.

The experiments show that performance depends on course structure and environmental conditions and that success on clean tracks does not imply robustness to currents. The fleet studies further show that local leader-follower coordination can reduce physical interaction, although its behavior depends on the selected policy and local progress estimates. Differences between controller-local progression and referee evaluation reinforce the need to keep navigation and scoring independent. Overall, Marine Race Arena provides a practical foundation for comparing underwater autonomy methods and studying perception-based control, disturbance rejection and multi-vehicle coordination.

REFERENCES

- [1] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [2] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in *Proceedings of the NeurIPS 2019 Competition and Demonstration Track*, ser. Proceedings of Machine Learning Research, vol. 123, 2020, pp. 77–89. [Online]. Available: <https://proceedings.mlr.press/v123/o-kelly20a.html>
- [3] S. Hoffmann, S. Sagmeister, T. Betz, J. Bongard, S. Büttner, D. Ebner, D. Esser, G. Jank, S. Goblirsch, A. Langmann, M. Leitenstern, L. Ögretmen, P. Pitschi, A.-K. Schwehn, C. Schröder, M. Weinmann, F. Werner, B. Lohmann, J. Betz, and M. Lienkamp, "Head-to-head autonomous racing at the limits of handling in the A2RL challenge," *arXiv preprint arXiv:2602.08571*, 2026, abu Dhabi Autonomous Racing League; preprint, not peer reviewed. [Online]. Available: <https://arxiv.org/abs/2602.08571>
- [4] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, "AlphaPilot: Autonomous drone racing," in *Robotics: Science and Systems (RSS)*, 2020. [Online]. Available: <https://www.roboticsproceedings.org/rss16/p081.html>
- [5] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.
- [6] E. Potokar, S. Ashford, M. Kaess, and J. G. Mangelson, "HoloOcean: An underwater robotics simulator," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3040–3046. [Online]. Available: <https://www.ri.cmu.edu/publications/holocean-an-underwater-robotics-simulator/>
- [7] M. von Benzon, F. F. Sørensen, E. Uth, J. Jouffroy, J. Liniger, and S. Pedersen, "An open-source benchmark simulator: Control of a BlueROV2 underwater robot," *Journal of Marine Science and Engineering*, vol. 10, no. 12, p. 1898, 2022.
- [8] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, "UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation," in *OCEANS 2016 MTS/IEEE Monterey*, 2016, pp. 1–8.
- [9] P. Cieślak, "Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface," in *OCEANS 2019 - Marseille*, 2019, pp. 1–6.
- [10] M. M. Zhang, W.-S. Choi, J. Herman, D. Davis, C. Vogt, M. McCarrin, Y. Vijay, D. Dutia, W. Lew, S. Peters, and B. Bingham, "DAVE aquatic virtual environment: Toward a general underwater robotics simulator," in *Proceedings of the IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, 2022, pp. 1–8. [Online]. Available: <https://arxiv.org/abs/2209.02862>
- [11] S. Chu, Z. Huang, Y. Li, M. Lin, D. Li, I. Carlucho, Y. R. Petillot, and C. Yang, "MarineGym: A high-performance reinforcement learning platform for underwater robotics," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 17 146–17 153. [Online]. Available: <https://ieeexplore.ieee.org/document/11246146>
- [12] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 78, 2017, pp. 1–16. [Online]. Available: <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
- [13] Autonomous Robotics Research Center, Technology Innovation Institute, "A2RL: Abu Dhabi autonomous racing league," <https://a2rl.io>, 2024, autonomous car and drone racing championship; accessed 2026-06-27.
- [14] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," *arXiv preprint arXiv:1606.01540*, 2016. [Online]. Available: <https://arxiv.org/abs/1606.01540>
- [15] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU-based physics simulation for robot learning," in *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.10470>
- [16] M. Stojanovic and J. Preisig, "Underwater acoustic communication channels: Propagation models and statistical characterization," *IEEE Communications Magazine*, vol. 47, no. 1, pp. 84–89, 2009.
- [17] Y. Yang, Y. Xiao, and T. Li, "A survey of autonomous underwater vehicle formation: Performance, formation control, and communication capability," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 815–841, 2021.
- [18] T. Yan, Z. Xu, and S. A. Gadsden, "Formation control of multiple autonomous underwater vehicles: A review," *Intelligence & Robotics*, vol. 3, no. 1, pp. 1–22, 2023.
- [19] A. Bedei, "HoloDroneCompetition: Marine Race Arena repository," <https://github.com/AndreaBedei/HoloDroneCompetition>, 2026.